

Computer Vision Homework 2

Name: NICKHOLAS

Student ID: M11515801

Department: NTUST CSIE

Write a program to do the following requirement.

I am using python 3.10 to implement all the image processing and histogram requirements.

Below are the results and brief explanations

(a) a binary image (threshold at 128)



Explanation:

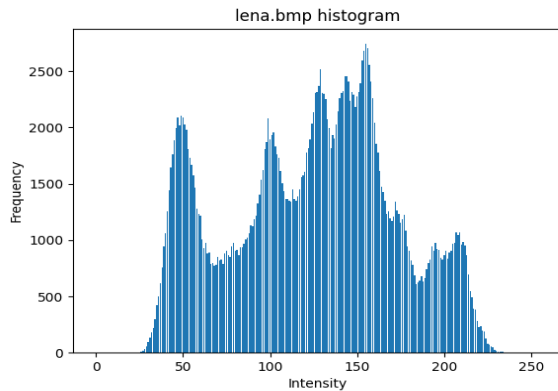
Using a nested loop to go through every pixel in the grayscale image. For each pixel, I compare its intensity value with the threshold value 128. If the pixel value is greater than or equal to 128, it is set to 255 (white). Otherwise, it is set to 0 (black). This creates a binary image by separating pixels into two groups based on the threshold.

```
# (a) a binary image (threshold at 128)
def binarize(img):
    h, w = img.shape
    binary = np.zeros((h, w), dtype=np.uint8)

    for i in range(h):
        for j in range(w):
            binary[i][j] = 255 if img[i][j] >= 128 else 0

    return binary
```

(b) a histogram



Explanation:

Using a nested loop to go through every pixel in the image and count how many times each intensity value (0–255) appears. I use an array of size 256 to store the frequency of each intensity. Each pixel increases the corresponding index in the histogram array by 1.

After computing the histogram using python, I save the result into an Excel file (histogram.xlsx). I also generate a bar chart image (histogram.png) to visualize the distribution of pixel intensities in lena.bmp.

```
# (b) a histogram
def compute_histogram(img):
    hist = [0] * 256

    for row in img:
        for pixel in row:
            hist[pixel] += 1

    return hist

# histogram
def save_histogram(hist):
    # excel spreadsheet
    wb = Workbook()
    ws = wb.active
    ws.title = "Histogram"

    ws.append(["Intensity", "Count"])
    for i, val in enumerate(hist):
        ws.append([i, val])
```

```

39
40     wb.save("histogram.xlsx")
41
42     # Save histogram image
43     plt.bar(range(256), hist)
44     plt.title("lena.bmp histogram")
45     plt.xlabel("Intensity")
46     plt.ylabel("Frequency")
47     plt.savefig("histogram.png")
48     plt.clf()
49

```

(c) connected components (regions with + at centroid, bounding box)



Explanation:

First, I use the binary image generated in part (a) (threshold 128). Then, I applied a connected component detection method using DFS. Using a nested loop, the code scans every pixel in the binary image. If a pixel is white (255) and has not been visited, the code starts a search with a stack to find all connected pixels belonging to the same region. Each pixel is marked as visited when it is pushed onto the stack, so it is never processed twice. In this assignment, I use 4-connectivity (up, down, left, right neighbors). All pixels belonging to the same region are grouped together and count the number of pixels in each region. If the count is greater than or equal to 500, the region is kept but if not, it is ignored.

```

def connected_components(binary):
    h, w = binary.shape
    visited = np.zeros((h, w), dtype=bool)
    components = []

    for i in range(h):
        for j in range(w):
            if binary[i][j] == 255 and not visited[i][j]:
                stack = [(i, j)]
                visited[i][j] = True
                pixels = []

                while stack:
                    x, y = stack.pop()
                    pixels.append((x, y))

                    for nx, ny in [(x-1,y),(x+1,y),(x,y-1),(x,y+1)]:
                        if 0 <= nx < h and 0 <= ny < w:
                            if binary[nx][ny] == 255 and not visited[nx][ny]:
                                visited[nx][ny] = True
                                stack.append((nx, ny))

                if len(pixels) >= 500:
                    components.append(pixels)

    return components

```

For each kept region:

- The region's pixels are filled with a color of their own.
- The bounding box is drawn from the minimum and maximum row and column of the region's pixels, using cv2.rectangle in the same color.
- A white "+" mark is drawn at the centroid.

```
def draw_components(img, components):
    color_img = np.stack([img, img, img], axis=2)

    for idx, comp in enumerate(components):
        color = get_color(idx, len(components))

        xs, ys = zip(*comp)
        min_x, max_x = min(xs), max(xs)
        min_y, max_y = min(ys), max(ys)

        for x, y in comp:
            color_img[x, y] = color

        cv2.rectangle(color_img, (min_y, min_x), (max_y, max_x), color, 2)

        cx = int(sum(xs) / len(xs))
        cy = int(sum(ys) / len(ys))

        cv2.line(color_img, (cy-10, cx), (cy+10, cx), (255,255,255), 2)
        cv2.line(color_img, (cy, cx-10), (cy, cx+10), (255,255,255), 2)

    return color_img
```

```
def get_color(idx, total):
    h = 6.0 * idx / max(total, 1)
    s, v = 0.78, 0.86
    i = int(h)
    f = h - i
    p = v * (1 - s)
    q = v * (1 - s * f)
    t = v * (1 - s * (1 - f))
    r, g, b = [(v, t, p), (q, v, p), (p, v, t),
                (p, q, v), (t, p, v), (v, p, q)][i % 6]
    return (int(b * 255), int(g * 255), int(r * 255))
```